

DAY-7

MILE STONE -1 (APPLICATION PROGRESS)

- **AUTHENTICATION:**

As APPLICATION DEVELOPERS we have 3 components:

1.FRONTEND

2.BACKEND

3.DATABASE

First part of the application to complete is authentication.It can be achieved through above three components.

- ❖ **DATABASE SETUP:**

DATABASE should be MONGODB,MYSQL,POSTGRES, SUPABASE,FIREBASE.

The database should be a cloud base DB

- **Cloud based DB:**

A **Cloud-Based Database** is a database that runs on a cloud platform and can be accessed from anywhere using an internet connection.

Why do we use a cloud-based database in an application?

A cloud-based database is used to store application data online so it is secure, accessible from anywhere, and easy to manage.

- **SCHEMA DEFINITIONS:**

A schema definition is the description of the database structure, including tables, columns (fields), data types, and the relationships between tables.

EXAMPLE:

In MONGODB :

{

ID:OBJECT

USERNAME:STRING

USERTYPE:ADMIN,USER

USERDETAILS:

```
{  
  AGE:  
  DOB:  
},  
  USERDAT:DATEOBJECT,  
  LASTMODIFIED:DATEOBJECT  
}
```

In MYSQL:

TABLERNAME:USER

COLUMNS:

USERNAME-STRING-256

USERTYPE-STRING-256

CREATEDAT-DATE-S(standard)

LASTMODIFIED-DATE-S

COURSES,REPORTS etc.

❖ FRONTEND:

HOMEPAGE- LOGIN/REGISTER

DASHBOARDS-ADMIN

DASHBOARDS-USER

Fronddend never communicate with the database for security basis.So we use backend

❖ BACKEND:

FRAMEWORK: JAVA SPRING BOOT 3

1. **JWT TOKENNIZATION**- It allows secure,authorized communication between frondend and database.

Backend goal is to connect to the database and get the required information.

- What is the difference between section based authorization and jwt based authorization?

Session-Based Authorization:

The server stores the user's session and identifies the user using a session ID.

JWT-Based Authorization:

The server issues a JWT token, and the client sends this token with each request. The server verifies the token instead of storing session data.

2.SERVICES:

@LOGIN

@REGISTER

@GETUSER

@MODIFY/UPDATE/DELETE USER

- **FLOW:**

USER -> HOMEPAGE-> LOGIN / REGISTER-> USEREMAIL,PASSWORD.

Validate the email->password encryption-> Reenter the password,validate if both match-> ENABLE SUBMIT .

It moves to @REGISTER service in the backend through API

API CONTRACT-> @REGISTER (frondend payload)

{

EMAIL:.,

PWD:ENCRYPTED SHA256.

JWTOKEN:(backend payload)

}

RESPONSE:

{

STATUS:OK/ERROR,

STATUSCODE:200 or 400 or 500 or 404.

MESSAGE: USER REGISTERED SUCCESSFULLY or COULDN'T CONNECT TO DB or
REGISTRATION FAILED or DATA ALREADY PRESENT.

}

This is the contract.

If OK-> ROUTE TO LOGIN

Else DISPLAY ERROR MESSAGE.

- API CONTRACT ->@LOGIN:

{

EMAIL:

PASSWORD:

JWT TOKEN

}

RESPONSE

{

STATUS: OK

STATUSCODE: 200 or 400 or 404.

MESSAGE: AUTHENTICATION SUCCESSFUL or FAILED.

If successful route to dashboard.

These is all about authentication in frontend.

BACKEND:

- @REGISTER->

Check the JWT Authentication is correct-> ACCEPT THE REQUEST

{

EMAIL:,

JWD:ENCRYPTED SHA256.

JWTTOKEN:

}

Connect to the Db-> Store the details and send the response

{

STATUS:OK/ERROR,

STATUSCODE:200 or 400 or 500 or 404.

MESSAGE: USER REGISTERED SUCCESSFULLY or COULDN'T CONNECT TO DB or
REGISTRATION FAILED or DATA ALREADY PRESENT.

}

- @LOGIN

ACCEPT THE REQUEST IF JWT AUTHENTICATION IS SUCCESSFUL

{

EMAIL:,

JWD:

JWTOKEN:

}

Connect to DB-> Search for user

Decode the password-> validate the password-> If MATCHES

{

STATUS:OK

STATUSCODE:200 or 400 or 500 or 404.

MESSAGE: USER REGISTERED SUCCESSFUL or FAILED.

These is all about the authentication in the backend.